

Dedication goes here.

Additional content can be written here.



# Acknowledgements

Acknowledgements go here.

d

*Acknowledgements*

# Contents

|          |                                        |          |
|----------|----------------------------------------|----------|
| <b>1</b> | <b>Abstract</b>                        | <b>g</b> |
| <b>1</b> | <b>Introduction</b>                    | <b>1</b> |
| <b>2</b> | <b>Pure Diffusion Model</b>            | <b>3</b> |
| 2.1      | 1D Transient Conduction Case . . . . . | 4        |
| 2.1.1    | Solver Comparison . . . . .            | 4        |
| 2.1.2    | Numerical Parameters Study . . . . .   | 6        |
| 2.1.3    | Model Validation . . . . .             | 8        |
| 2.2      | 2D Transient Conduction Case . . . . . | 9        |
| 2.2.1    | Numerical Parameter Study . . . . .    | 11       |
| 2.2.2    | Model Validation . . . . .             | 12       |



## Abstract

The objective of this project is to numerically solve the equations of fluid motion with heat transfer, i.e., the Navier-Stokes equations coupled with the energy conservation equation, in a multidimensional manner (CFD & HT methods). The methodology will be based on finite-volume techniques, using the fractional-step method to solve the coupling between the pressure and velocity fields, i.e., the conservation of mass and momentum equations. In a further step, the energy conservation equation will be incorporated, enabling the study of convection heat transfer problems. The analysis will begin with studies of conduction heat transfer problems (i.e., 4-materials problem). In a second step, a given velocity field will be incorporated to study the numerical diffusion problem in the solution of the convection-diffusion equation (i.e., Smith-Hutton problem). In the third step, a fluid motion solver for laminar flows will be developed, i.e., the mass and momentum conservation equations; finally, the energy conservation equation will be added to the fluid solver. The computational codes will be verified through grid-refinement studies in both space and time, using corresponding benchmark cases at each step. The investigation of the selected problems will include parametric studies considering variations in geometry, thermophysical properties, and boundary conditions.





# Introduction

This report will cover the development of numerical tools for the resolution of Convection-Diffusion problems while consolidating the knowledge related to the transport phenomena, and the application of numerical methods for their resolution. The models implemented in this project will use a finite volume method to discretize the analyzed domain. In addition, the goal of this project implies the development of a robust and versatile code structure to ensure the models can be easily applied to any problem configuration. This chapter will introduce the main concepts related to the governing equations and the discretization method. Furthermore, the development of the models will be separated into a first portion focused on a pure diffusion model, and a second one implementing the complete convection-diffusion model. The code structure will be explained within each model implementation as a reflection of the evolution of the model throughout the development of this project.

When studying transport phenomena, there are two main mechanisms that govern the propagation of a transport variable  $\phi$ . These are known as **Convection** and **Diffusion**, and classify the phenomena depending on what causes the transport of the transport variable  $\phi$ . Diffusion refers to transport caused by molecular movement within the body, while Convection refers to transport caused by bulk movement of the body. The governing transport equation for this phenomena is given by Eq. 1.1, where each element can be discretized as shown in Eqs. 1.2, 1.3, 1.4. The discretization considers a rectangular prism control volume and is done considering a 2D mesh, keeping in mind

that the 1D case ignores the elements for south and north neighbors. This discretization model will be the basis for the implementation shown in the following chapters, with appropriate clarifications for the nature of  $\phi$  in each case.

$$\rho \frac{\partial \phi}{\partial t} + \nabla(\rho v \phi) = \Gamma \nabla \phi + S_\phi \quad (1.1)$$

$$\rho \frac{\partial \phi}{\partial t} = \rho \frac{\phi_P - \phi_P^0}{\Delta t} \quad (1.2)$$

$$\nabla(\rho v \phi) = \rho(v_W \phi_W S_W - v_E \phi_E S_E + v_S \phi_S S_S - v_N \phi_N S_N) \quad (1.3)$$

$$\Gamma \nabla \phi = \Gamma_W \frac{\phi_W - \phi_P}{d_{PW}} - \Gamma_E \frac{\phi_P - \phi_E}{d_{PE}} + \Gamma_S \frac{\phi_S - \phi_P}{d_{PS}} - \Gamma_N \frac{\phi_P - \phi_N}{d_{PN}} \quad (1.4)$$

Is this too simplified? Should I include the integration process?

## Pure Diffusion Model

This chapter will explain the implementation of a pure diffusion model. For a pure diffusion model the governing transport equation can be reduced to the expression shown in Eq. 2.1, with the corresponding discretized version shown in Eq. 2.2. For the case of the thermal conduction model being implemented in this chapter, the equivalent expressions using Temperature as the transport variable and including the relevant physical parameters are shown in Eqs. (2.3), (2.4).

$$\rho \frac{\partial \phi}{\partial t} = \Gamma \nabla \phi + S_\phi \quad (2.1)$$

$$\rho c_P V_P \frac{\phi_P - \phi_P^0}{\Delta t} = \Gamma_W \frac{\phi_W - \phi_P}{d_{PW}} + \Gamma_E \frac{\phi_P - \phi_E}{d_{PE}} + \Gamma_S \frac{\phi_S - \phi_P}{d_{PS}} + \Gamma_N \frac{\phi_P - \phi_N}{d_{PN}} + S_\phi \quad (2.2)$$

$$\rho c_P V_P \frac{\partial T}{\partial t} = \lambda \nabla T + S_V \quad (2.3)$$

$$\rho c_P V_P \frac{T_P - T_P^0}{\Delta t} = \lambda_W S_W \frac{T_W - T_P}{d_{PW}} + \lambda_E S_E \frac{T_P - T_E}{d_{PE}} + \lambda_S S_S \frac{T_S - T_P}{d_{PS}} + \lambda_N S_N \frac{T_P - T_N}{d_{PN}} \quad (2.4)$$

## 2.1 1D Transient Conduction Case

In this section, the implementation of a C++ code for the resolution of a one-dimensional transient-state heat conduction model is described. The model works using a configuration file that includes multiple parameters that define the simulation. A summary of all the different parameters included in the file is shown in Table 2.1. As shown in the table, the configuration parameters have been separated into different groups. Using this model, a study will be made that analyzes the effect different parameters can have on resolution and performance. The main focus of this study will be to analyze the effect of some of the numerical parameters.

**Physical** includes the initial temperature and a list of the available materials and their thermophysical properties. **Geometrical** includes the geometric parameters; the model can only support rectangular shapes in this implementation and changes will be made when more complex geometries are studied. Sections must be defined with a material, the number of nodes for that specific section  $N_s$ , beginning and ending coordinates  $x_0$  and  $x_1$ , and internal heat generation  $q_V$ . **Numerical** includes the numerical parameters; the model currently has both the Gauss-Seidel (GS) and Conjugate Gradient (CG) solvers implemented and both will be compared as a first step before performing the relevant studies discussed ahead. **Mesh** includes the option to choose from four different algorithms, as well as all relevant parameters for each of these alternatives. **Boundary Conditions** includes an array describing the boundaries for the model and can currently support Dirichlet, Neumann, and Convection boundary conditions. In addition, it supports variable temperatures for Dirichlet boundary conditions.

### 2.1.1 Solver Comparison

As mentioned above, the model described in this report has both GS and CG solvers implemented. Only one will be used to perform the numerical parameter studies shown below; as such, relevant metrics need to be determined to compare both solvers. For this purpose, the number of iterations and residues for each time step will be used to

**Table 2.1:** Configuration file structure.

| Group            | Parameter       | Description                                                                                                                             |
|------------------|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Physical Data    | T0              | Initial value for temperature map.                                                                                                      |
|                  | materials       | List of vectors with thermophysical properties corresponding to each material.                                                          |
| Geometrical Data | width<br>height | Dimension in secondary axis.<br>Dimension in secondary axis.                                                                            |
|                  | sections        | List of vectors indicating the distribution of materials in the simulation. Also includes amount of nodes and internal heat generation. |
| Numerical Data   | scheme          | Scheme for simulation.                                                                                                                  |
|                  | solver          | Solver algorithm for simulation.                                                                                                        |
|                  | maxIterations   | Limit of iterations per timestep.                                                                                                       |
|                  | endTime         | Last instant of the simulation.                                                                                                         |
|                  | timeStep        | Time increment after each instant.                                                                                                      |
| Mesh Data        | tolNumeric      | Tolerance for numerical solver.                                                                                                         |
|                  | tolTemporal     | Tolerance for steady-state convergence.                                                                                                 |
| Mesh Data        | meshAlgorithm   | Algorithm for mesh generation.                                                                                                          |
|                  | strength        | Cubic algebraic mesh parameter.                                                                                                         |
|                  | centering       | Cubic algebraic mesh parameter.                                                                                                         |
|                  | kappa<br>delta  | Power law mesh parameter.<br>Hyperbolic tangent mesh parameter.                                                                         |
| Boundary Data    | boundaries      | List of vectors describing the boundary conditions on each side.                                                                        |

compare both solutions. The base simulation used in this study will follow the case described in *Exercise 1: 1D transient conduction case*, with the addition of an internal heat source for a better comparison with the steady-state analytical solution.

This simulation will use  $N = 100$ ,  $\Delta t = 0.5$  and a Crank-Nicolson scheme with a uniform mesh. The results for both cases are shown in Figure 2.1. In both metrics, it can be seen that CG outperforms GS by a considerable margin; converging faster and obtaining lower residue. As such, CG will be used as the linear solver for the following studies. In addition, the computation time was also slightly faster for CG compared to GS with a total of 0.60 and 0.77 seconds, respectively; although this is the least relevant comparison.

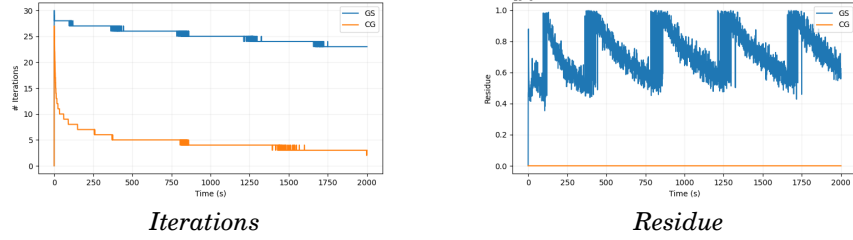


Figure 2.1: Solver comparisons.

### 2.1.2 Numerical Parameters Study

For this test, the time step  $\Delta t$  and the number of nodes  $N$  are modified to analyze the number of iterations and compare them with the analytical solution. Both parameters will be independently swept across their respective range of values, while keeping the other parameter at a constant value. A summary of the values for each one of these parameters is shown in Table 2.2, highlighting the values used for the base case. In addition, the explicit scheme has an imposed limit for the value of  $\Delta t$  to ensure convergence, given by the expression shown in Eq. 2.5.

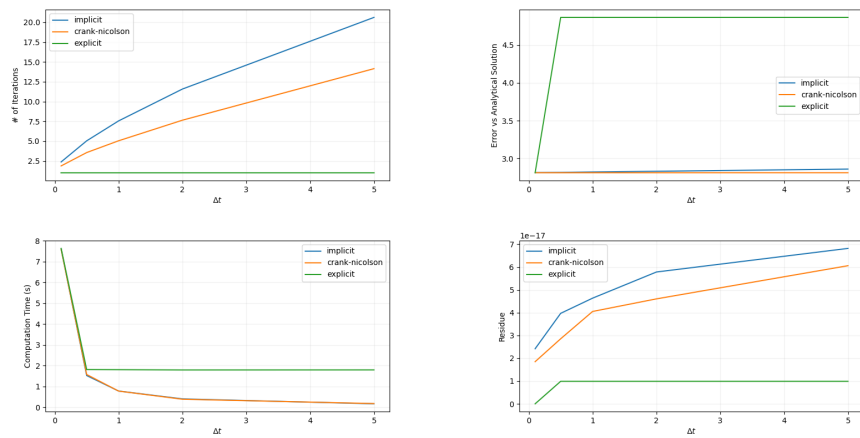
Table 2.2: Detail of numerical parameters to be studied.

| Parameter  | Values                          |
|------------|---------------------------------|
| $\Delta t$ | 0.1, <b>0.5</b> , 1.0, 2.0, 5.0 |
| $N$        | 50, <b>100</b> , 200, 400       |

$$\Delta t_{max} = \frac{\Delta x^2}{\alpha} \quad (2.5)$$

**$\Delta t$  Sweep** The results obtained from the tests explained above are shown in Figure 2.2. As expected, the number of iterations remains unchanged for the explicit scheme, as it is meant to be a direct calculation. On the other hand, both the crank-nicolson and implicit schemes require more iterations per time step as it increases. Compared to the analytical solution, the explicit scheme shows a significant error for  $0.1 < \Delta t$ , while a small increase could be seen for the crank-nicolson and implicit schemes. Overall, the crank-nicolson scheme has shown better performance in terms of number of iterations needed and precision compared to the analytical solution. Furthermore, the results for

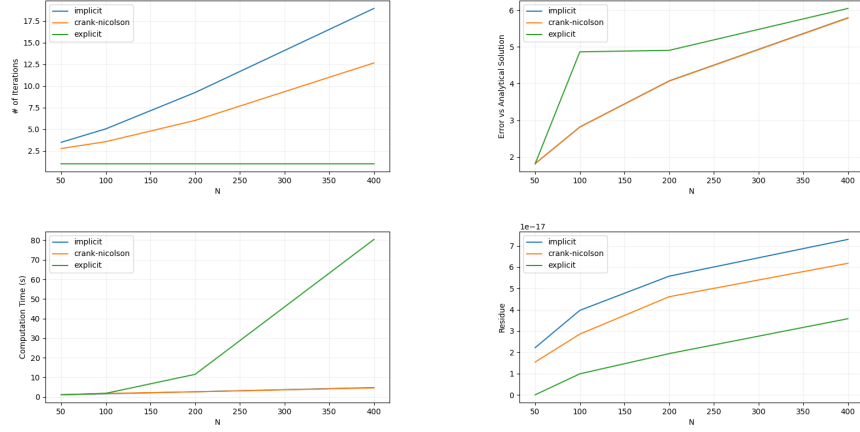
explicit seem to stagnate at a constant value after the lowest time step value tested, suggesting that the limit set by Eq. 2.5 has been exceeded. However, the high error suggests that that expression should be revised for a more conservative estimation. The computation time is seen to be similar between the crank-nicolson and implicit schemes, with a small advantage for the implicit scheme.



**Figure 2.2:**  $\Delta t$  sweep results.

No me gusta como se ve el subsubsection, cambiar la función para que deje un espacio después del título como en section (pero sin numerar)

*N Sweep* The results obtained for the tests explained above are shown in Figure 2.3. Similarly to the previous case, the explicit scheme is a direct calculation and does not require more iterations. In addition, showing an increase in the number of iterations for the crank-nicolson and implicit schemes, suggesting better performance for crank-nicolson as the mesh gets finer. Compared to the analytical solution, the explicit scheme still showed higher values compared to the crank-nicolson and implicit schemes; however, it seems to get closer as the number of nodes increases. In terms of computation time, both the crank-nicolson and implicit schemes saw a slight increase, while the explicit scheme saw a major difference as the mesh got finer.

**Figure 2.3:**  $N$  sweep results.**Table 2.3:** Cases for validation.

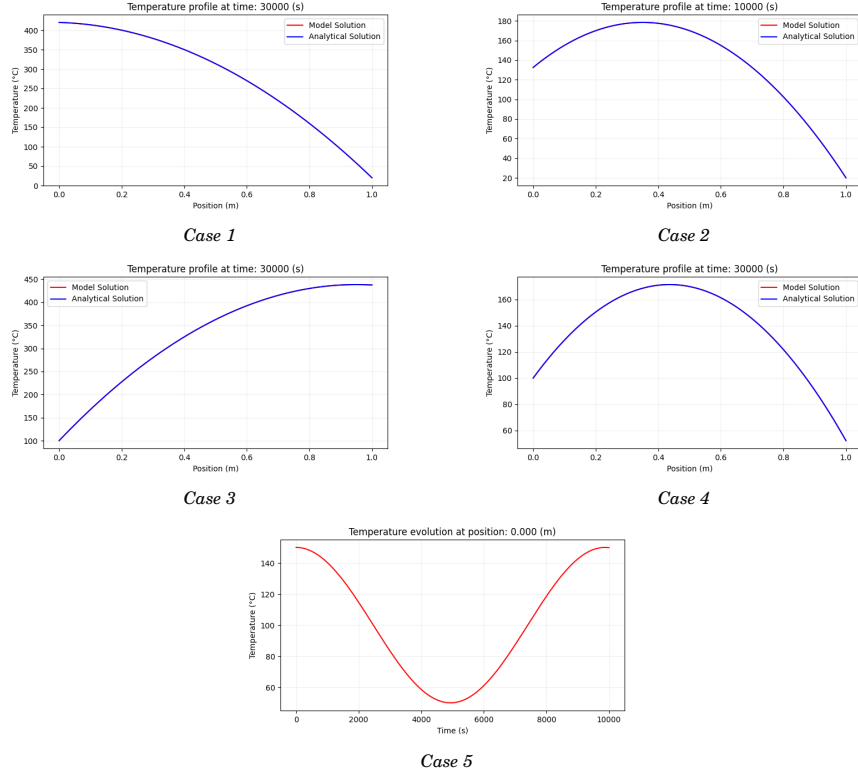
| Case | Boundary Conditions                                   |
|------|-------------------------------------------------------|
| 1    | Neumann-Dirichlet                                     |
| 2    | Convection-Dirichlet                                  |
| 3    | Dirichlet-Neumann                                     |
| 4    | Dirichlet - Convection                                |
| 5    | $T(t) = 100 + 50 \cos\left(\frac{0.01t}{5\pi}\right)$ |

### 2.1.3 Model Validation

In addition to the Dirichlet boundary condition established in the problem statement, Neumann and Convection boundary conditions have also been implemented in the model. Furthermore, support for variable temperatures for Dirichlet boundary conditions was implemented. To validate this implementation, a comparison will be made with the analytical solution for a steady-state scenario with the transient-state model once it reaches equilibrium. A summary of the pairs of boundary conditions used for the tests is shown in Table 2.3, while comparisons with the steady state solution are shown in Figure 2.4. An additional case is included to test the proper implementation of the variable temperature Dirichlet boundary condition. As shown, all pairs evaluated accurately matched the analytical solution. Furthermore, the configuration file can now provide mathematical expressions instead of constant values when defining Dirichlet



boundary conditions, as shown by the temperature profile. This capability can now easily be scaled up towards other functions in the model.



**Figure 2.4:** Comparison of model results with analytical solutions and implementation of mathematical expression reading for Dirichlet boundary condition.

## 2.2 2D Transient Conduction Case

In this section, the implementation of a C++ code for the resolution of a two-dimensional transient state heat conduction model is described. This model has several modifications compared to the previous one, mainly reflected in the format for the configuration file. Table 2.4 summarizes the new parameters accepted in the configuration file. As shown, probes have now been added as a configurable parameter to allow the user to choose which data is being saved. In addition, refinement can now be defined independently

for each axis and in separate regions to allow the user better control over the mesh. Furthermore, a Medic class was also included to handle diagnostics and validation of energy conservation in the system. This class is also used as the central tool for debugging as it can easily be modified to log any information relevant to the model.

**Table 2.4:** Configuration file structure.

| Group            | Parameter     | Description                                                                                                                             |
|------------------|---------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Numerical Data   | scheme        | Scheme selection for simulation.                                                                                                        |
|                  | solver        | Solver algorithm for simulation.                                                                                                        |
|                  | tolNumeric    | Tolerance for the numerical solver.                                                                                                     |
|                  | tolTemporal   | Tolerance for the steady-state convergence.                                                                                             |
|                  | maxIterations | Limit of iterations per timestep.                                                                                                       |
|                  | endTime       | Last instant of the simulation.                                                                                                         |
|                  | timeStep      | Time increment after each instant.                                                                                                      |
| Physical Data    | T0            | Initial value for temperature map.                                                                                                      |
|                  | materials     | List of vectors with thermophysical properties corresponding to each material.                                                          |
| Geometrical Data | width         | Dimension in the secondary axis.                                                                                                        |
|                  | sections      | List of vectors indicating the distribution of materials in the simulation. Also includes amount of nodes and internal heat generation. |
| Mesh Data        | strength      | Cubic algebraic mesh parameter.                                                                                                         |
|                  | centering     | Cubic algebraic mesh parameter.                                                                                                         |
|                  | kappa         | Power law mesh parameter.                                                                                                               |
|                  | delta         | Hyperbolic tangent mesh parameter.                                                                                                      |
|                  | meshAlgorithm | Algorithm for mesh generation.                                                                                                          |
|                  | N             | List specifying total number of nodes for each axis.                                                                                    |
| Boundary Data    | refinement    | List of vectors specifying refinement configuration for each axis.                                                                      |
|                  | boundaries    | List of vectors specifying the boundary conditions on each side.                                                                        |
| Probe Data       | probes        | List of vectors specifying the probes configured for the simulation.                                                                    |

### 2.2.1 Numerical Parameter Study

For this test, timestep  $\Delta t$  and number of nodes  $N$  are modified to analyze their impact on the number of iterations and total residue. Both parameters will be independently swept across their respective range of values while keeping the other parameter at a constant value. A summary of the values for each of these parameters is shown in Table 2.5, highlighting the values used for the base case. In addition, the explicit scheme still requires an imposed limit for the value of  $\Delta t$ , however the expression for the 2D case changes as described by Eq. 2.6. Contrary to the results shown for the 1D case, there will be no comparison of error with the analytical solution or computation time. This study numerical parameter study will focus on comparing the number of iterations and total residue.

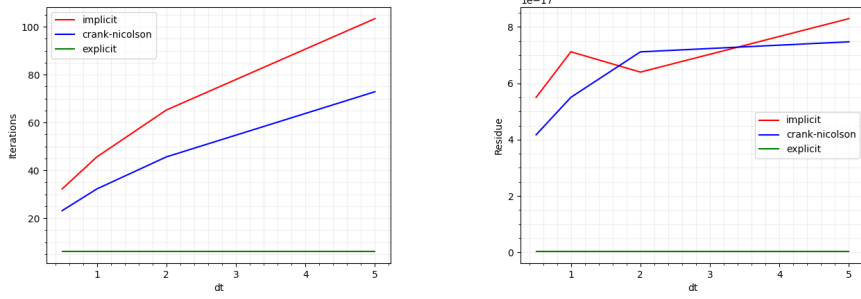
**Table 2.5:** Value ranges for numerical parameter study.

| Parameter  | Values                          |
|------------|---------------------------------|
| $\Delta t$ | 0.1, 0.5, <b>1.0</b> , 2.0, 5.0 |
| $N$        | 50, <b>100</b> , 200, 400       |

$$\Delta t_{max} = \frac{\Delta x^2 \Delta y^2}{2\alpha(\Delta x^2 + \Delta y^2)} \quad (2.6)$$

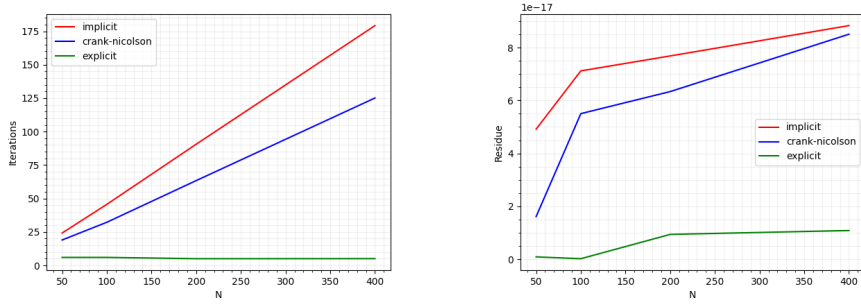
*$\Delta t$  Study* The results obtained from the tests explained above are shown in Figure 2.5. As shown, the number of iterations and total residue follow a similar trend to the one seen in the 1D case, increasing with  $\Delta t$  for the implicit and crank-nicolson schemes. Showing a better performance for the crank-nicolson scheme on both metrics. On the other hand, the explicit scheme did not present any changes during the timestep study. This is due to the timestep being limited by the expression discussed above; which in this case happened for all analyzed cases.

*$N$  Study* The results obtained from the tests explained above are shown in Figure 2.6. Just like in the previous test, the presented plots show a similar trend to the one seen in the previous scenario, showing an increase in number of iterations and residue for most cases. Furthermore, the implicit scheme showed higher sensitivity to increases in  $N$  compared to the crank-nicolson and explicit schemes; with the explicit scheme showing no change at all. On the other hand, while residue was significantly higher for implicit on coarser meshes, the difference between the implicit and crank-nicolson



**Figure 2.5:**  $\Delta t$  sweep results.

schemes became smaller as  $N$  increased; with the explicit scheme showing a slight increase in the beginning but less of an impact as  $N$  kept increasing.



**Figure 2.6:**  $N$  sweep results.

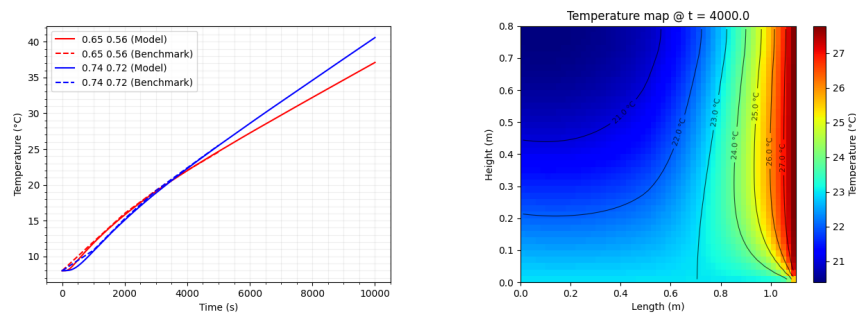
### 2.2.2 Model Validation

In the same way as done in Section 2.1, boundary conditions were independently analyzed to validate proper implementation in the model by comparing it to the steady-state analytical solution. To simulate this behaviour, an adiabatic Neumann boundary condition ( $\Delta\phi = 0$ ) was applied on the faces of the secondary axis in each test. On the other hand, the boundary conditions for the primary axis in the test were configured following the same testing plan described in Section 2.1. The results obtained in this

validation successfully replicated the expected behaviour and produced the same plots as those seen in Figure 2.4 for each axis. Furthermore, a comparison with the *Exercise 3: 4 Materials Case* will be used as a benchmark to complete the validation of the model.

Creo que sería redundante volver a poner las mismas gráficas que puse en la otra sección para los boundary conditions y debería ser suficiente con la validación con el caso. Debería dejar de mencionarlo en general?.

*4 Materials Case* The results obtained for *Exercise 3: 4 Materials Case* are shown in Fig. 2.7. As mentioned above, this is a transient simulation and there is no analytical solution available to compare the results with. Instead, this case provides reference values for two separate points at different instants of the simulation. As seen in the results, the model was able to match the values provided for points [0.65,0.56] and [0.74,0.72]; as well as showing a temperature distribution consistent with the provided time instants.



**Figure 2.7:** Comparison of model results with results provided in problem statement.

Debería incluir el resto de temperature maps? Siento que es llenarlo con más de lo mismo.